

Chapter V

Mandatory part - cub3D

Program Name	cub3D
Files to Submit	All your files
Makefile	all, clean, fclean, re, bonus
Arguments	a map in format *.cub
External Function	• open, close, read, write, printf, malloc, free, perror, strerror, exit, gettimeofday. • All functions of the math library (-lm man man 3 math). • gettimeofday() • All functions of the MinilibX library.
Libft authorized	Yes
Description	You must create a "realistic" 3D graphical representation of the inside of a maze from a first-person perspective. You have to create this representation using the ray-casting principles mentioned earlier.

The constraints are as follows:

- You **must** use the miniLibX. Either the version that is available on the operating system, or from its sources. If you choose to work with the sources, you will need to apply the same rules for your libft as those written above in **Common Instructions** part.
- The management of your window must remain smooth: changing to another window, minimizing, etc.
- Display different wall textures (the choice is yours) that vary depending on which side the wall is facing (North, South, East, West).

- Your program must be able to set the floor and ceiling colors to two different ones.
- The program displays the image in a window and respects the following rules:
 - The left and right arrow keys of the keyboard must allow you to look left and right in the maze.
 - The W, A, S, and D keys must allow you to move the point of view through the maze.
 - Pressing **ESC** must close the window and quit the program cleanly.
 - Clicking on the red cross on the window's frame must close the window and quit the program cleanly.
 - The use of `images` of the `minilibX` library is strongly recommended.
- Your program must take as a first argument a scene description file with the `.cub` extension.
 - The map must be composed of only 6 possible characters: **0** for an empty space, **1** for a wall, and **N,S,E** or **W** for the player's start position and spawning orientation.

This is a simple valid map:

```
111111
100101
101001
1100N1
111111
```

- The map must be closed/surrounded by walls, if not the program must return an error.
- Except for the map content, each type of element can be separated by one or more empty lines.
- Except for the map content which always has to be the last, each type of element can be set in any order in the file.
- Except for the map, each type of information from an element can be separated by one or more spaces.
- The map must be parsed as it looks in the file. Spaces are a valid part of the map and are up to you to handle. You must be able to parse any kind of map, as long as it respects the rules of the map.

- Except for the map, each element must begin with its type identifier (composed by one or two characters), followed by its specific information in a strict order:

- * North texture:

```
NO ./path_to_the_north_texture
```

- identifier: **NO**
- path to the north texture

- * South texture:

```
SO ./path_to_the_south_texture
```

- identifier: **SO**
- path to the south texture

- * West texture:

```
WE ./path_to_the_west_texture
```

- identifier: **WE**
- path to the west texture

- * East texture:

```
EA ./path_to_the_east_texture
```

- identifier: **EA**
- path to the east texture

- * Floor color:

```
F 220,100,0
```

- identifier: **F**
- R,G,B colors in range [0,255]: **0, 255, 255**

* Ceiling color:

```
C 225,30,0
```

- identifier: **C**
- R,G,B colors in range [0,255]: **0, 255, 255**

○ Example of the mandatory part with a minimalist **.cub** scene:

```
NO ./path_to_the_north_texture
SO ./path_to_the_south_texture
WE ./path_to_the_west_texture
EA ./path_to_the_east_texture

F 220,100,0
C 225,30,0

11111111111111111111111111111111
10000000001100000000000001
10110000011100000000000001
10010000000000000000000001
11111111101100000111000000000001
100000000001100000111011111111111
11110111111111011100000010001
11110111111111011101010010001
110000001101011100000010001
100000000000000001100000010001
100000000000000001101010010001
1100000111010101111011110N0111
11110111 1110101 101111010001
11111111 1111111 111111111111
```

○ If any misconfiguration of any kind is encountered in the file, the program must exit properly and return "Error\n" followed by an explicit error message of your choice.

Chapter VI

Readme Requirements

A README.md file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.

The README.md must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A “**Description**” section that clearly presents the project, including its goal and a brief overview.
- An “**Instructions**” section containing any relevant information about compilation, installation, and/or execution.
- A “**Resources**” section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➡ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.



Your README must be written in English.